

MetroRent Price Estimator

Project Report

Prepared by: Harika

Abstract

This report presents a multi-city monthly asking-rent estimator built from 21,691 public rental listings. I harmonised two source schemas, applied property and rent validity rules, removed duplicate listing signatures, and retained 12,201 rows for model development.

The final model achieved INR 11,652 mean absolute error on 1,831 held-out listings compared with INR 22,122 for a city-median baseline. A separate calibration partition produced an interval with 89.7 percent test coverage. Five experiments explain dataset retention, baseline value, city-level error, interval calibration, and city rent differences.

Keywords: rental price modelling; gradient boosting; calibration; baseline comparison; Indian housing

Report structure

The report presents the problem, source data, preparation method, evaluation design, five evidence-based experiments, limitations, reproducibility details, and conclusion.

1. Project overview and problem statement

MetroRent estimates monthly asking rent for listings across Bangalore, Mumbai, Nagpur, New Delhi, and Pune. I built the project because city averages cannot represent the interaction between locality, property size, bedrooms, bathrooms, furnishing, and property type.

The project tests whether a reproducible model improves on a transparent city-median baseline and whether a separate calibration split can produce a useful uncertainty interval. It is a listing-comparison model, not a formal valuation or negotiation instruction.

2. Data sources and preparation

Two MIT-licensed Kaggle sources contribute 21,691 raw rental listings. I harmonised city and locality names, converted numerical fields, removed impossible configurations, capped implausible values, and removed duplicate listing signatures.

The final table contains 12,201 model rows across five cities and 2,102 localities. The split uses 8,540 training rows, 1,830 calibration rows, and 1,831 final test rows.

Stage	Rows
Raw listings	21,691
Prepared model table	12,201
Training	8,540
Calibration	1,830
Test	1,831

3. Model and evaluation design

The model combines numerical and categorical listing attributes in a scikit-learn pipeline and trains a gradient-boosting regressor. A city-median predictor is the required baseline. Model selection and fitting are separated from final test evaluation.

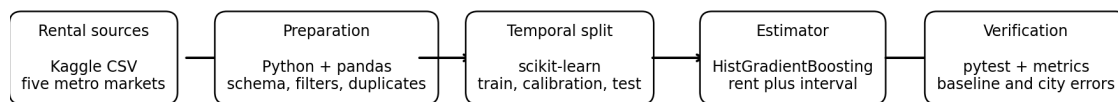
A conformal-style interval radius is estimated from absolute calibration errors and applied unchanged to the test set. I report MAE, RMSE, R-squared, interval coverage, and city-level error.

4. End-to-end model architecture

The architecture separates source alignment, preparation, data splitting, model fitting and verification. Python and pandas resolve the two source schemas and remove invalid or duplicate listings before scikit-learn receives the modelling table.

Training, calibration and test rows remain distinct. The saved estimator returns a point estimate and a calibrated interval, while evaluation JSON and figures preserve the baseline, city-level error and coverage results used in the report.

MetroRent end-to-end model architecture



Architecture diagram. MetroRent source, preparation, model and evaluation stages.

Stage design: Every stage names the technology and the artefact handed to the next stage.

Leakage control: The calibration and test partitions are not reused for model fitting.

Output: The prediction interface returns a rent estimate with an interval rather than presenting false precision.

5. Automated model test execution

I ran the current repository test suite after updating the report. Three tests passed. The checks cover prepared-data rules, saved-model inference and ordering of the lower estimate, point estimate and upper estimate for an unseen listing.

The tests complement model metrics. MAE describes predictive error across the holdout, while the test suite verifies software behaviour and asset contracts that a single score cannot detect.

```
Actual model test execution

$ .venv/bin/pytest -q
tests/test_data_rules.py .. [ 67%]
tests/test_prediction.py . [100%]

3 passed in 2.38s

Validated: prepared-data rules, saved-model inference
and prediction interval ordering on unseen input.
```

Test execution evidence. Actual MetroRent pytest execution for data and prediction contracts.

Execution: Three tests completed successfully in 2.38 seconds.

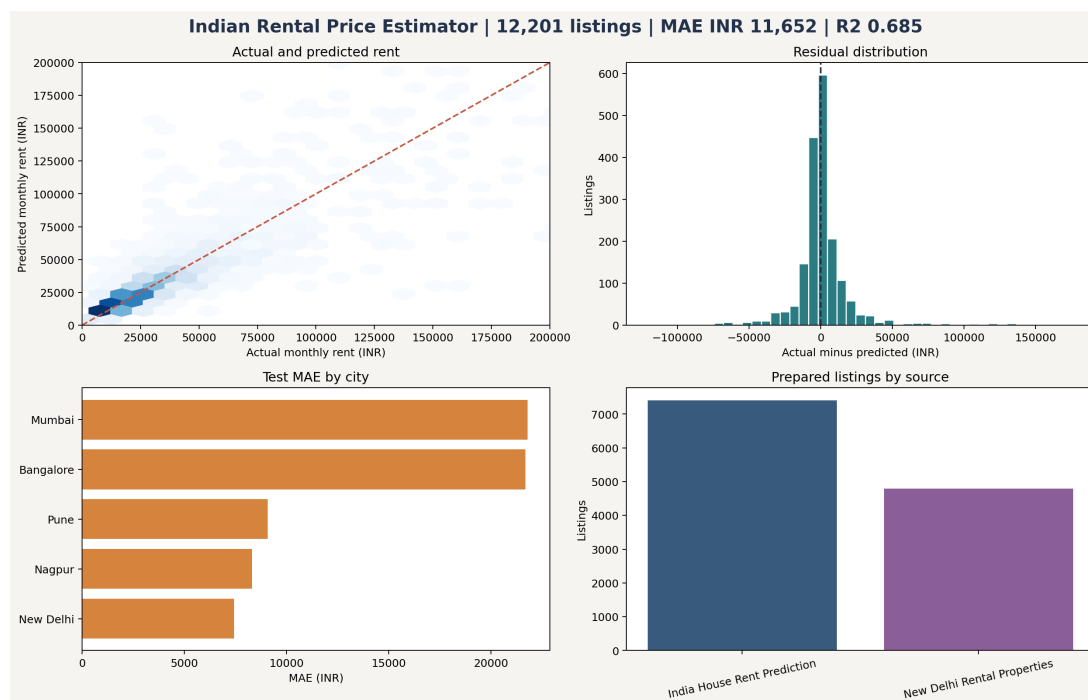
Scenarios: The suite includes valid inference and interval ordering, not only file-existence checks.

Boundary: Distribution shift and housing-market changes require future monitoring beyond unit tests.

6. Evaluation dashboard and decision interpretation

The evaluation dashboard combines the model-versus-baseline comparison, city errors and prediction interval evidence. It allows the headline MAE to be interpreted beside the large differences between metro markets.

The dashboard is not a rental valuation service. Its purpose is to make model performance, uncertainty and data scope visible before a user considers an individual estimate.



Evaluation dashboard. MetroRent baseline, city and interval results.

Question: Does the selected model improve on a transparent baseline while maintaining useful interval coverage?

Observed result: The model reduces MAE from INR 22,122 to INR 11,652 and the interval covers 89.7 percent of test rents.

Decision use: City-level error remains visible so the aggregate improvement is not interpreted as equal performance everywhere.

7. Experiment 1: data flow

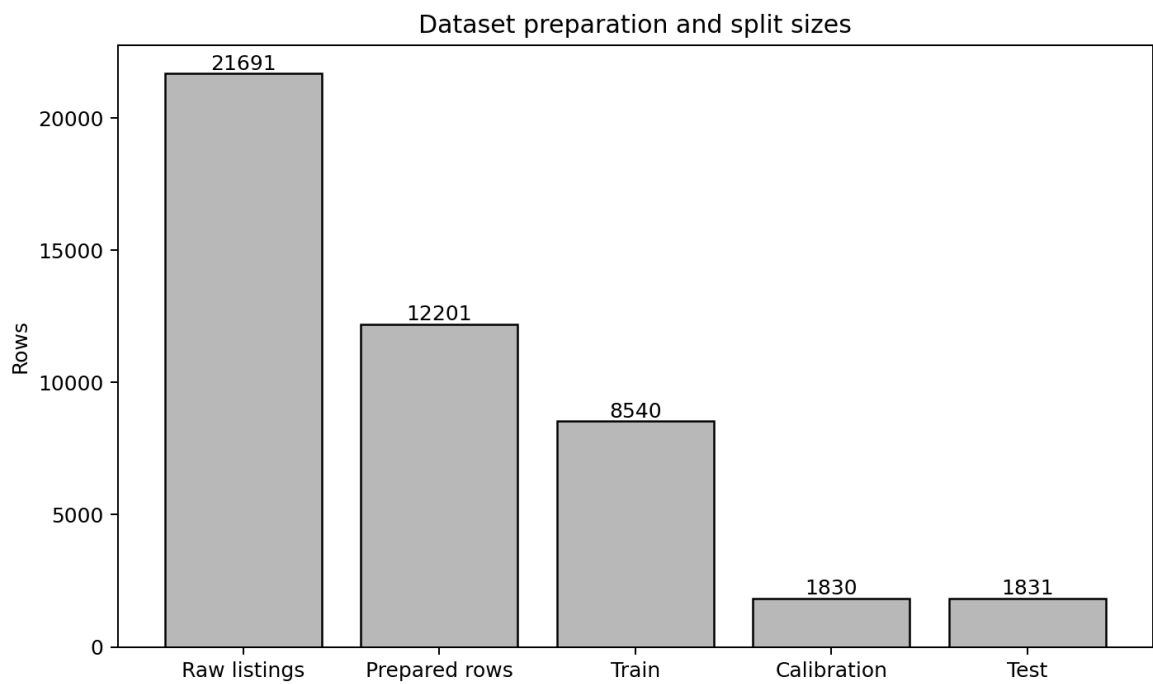


Figure 1. Raw, prepared, training, calibration, and test row counts.

What I tested: How much source data remains after quality rules and how the final rows are allocated.

What the graph shows: 12,201 of 21,691 raw listings remain, with independent calibration and test partitions.

Conclusion: The preparation removes substantial source noise while keeping enough rows for multi-city evaluation.

8. Experiment 2: baseline comparison

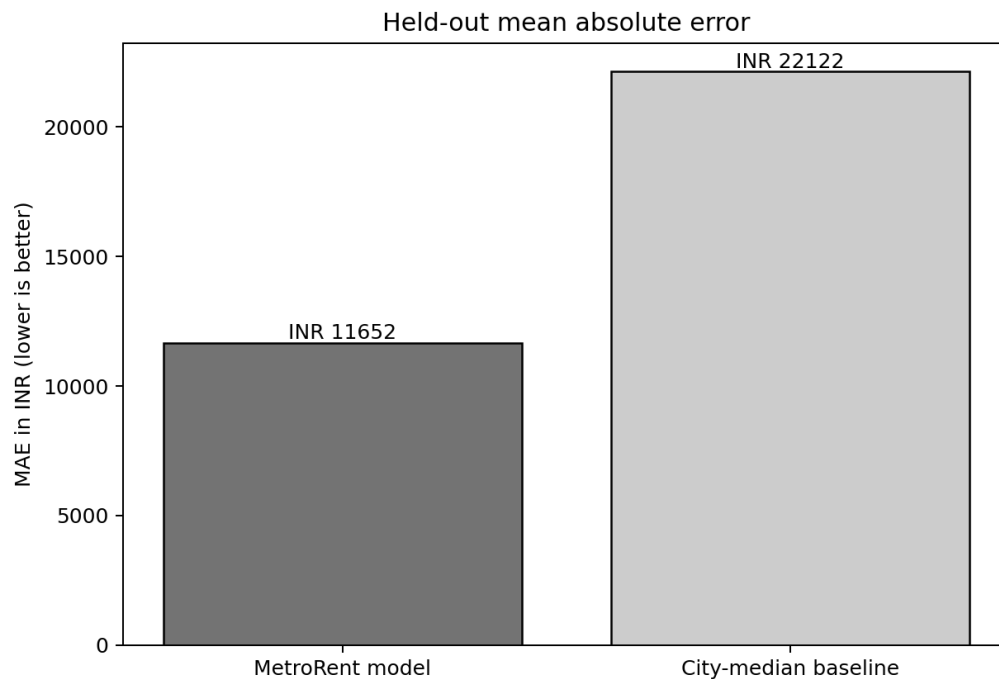


Figure 2. Held-out MAE for MetroRent and the city-median baseline.

What I tested: Whether the learned model adds value beyond a simple city-level typical rent.

What the graph shows: Model MAE is INR 11,652 compared with INR 22,122 for the baseline.

Conclusion: Property and locality features materially improve average test error, although the remaining INR error is still substantial.

9. Experiment 3: city error

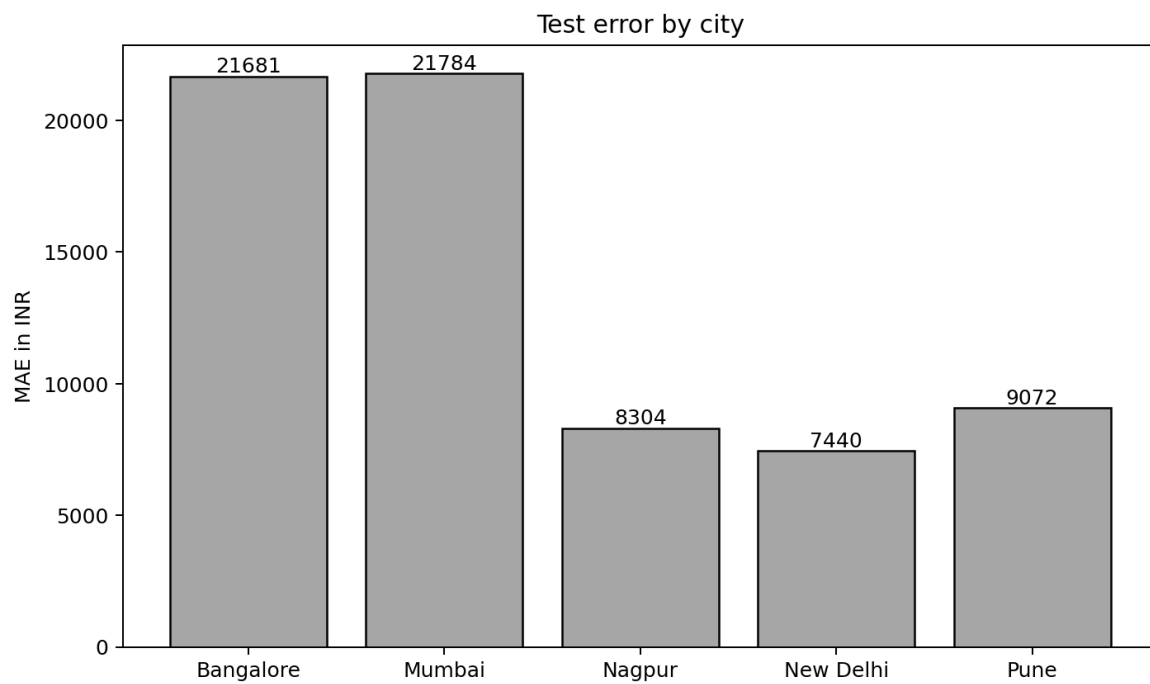


Figure 3. Test MAE by city.

What I tested: Whether one aggregate error represents all five cities.

What the graph shows: Mumbai and Bangalore are the hardest, while New Delhi has the lowest MAE in this test split.

Conclusion: Performance is geographically uneven and should always be reported with city-level sample sizes.

10. Experiment 4: interval calibration

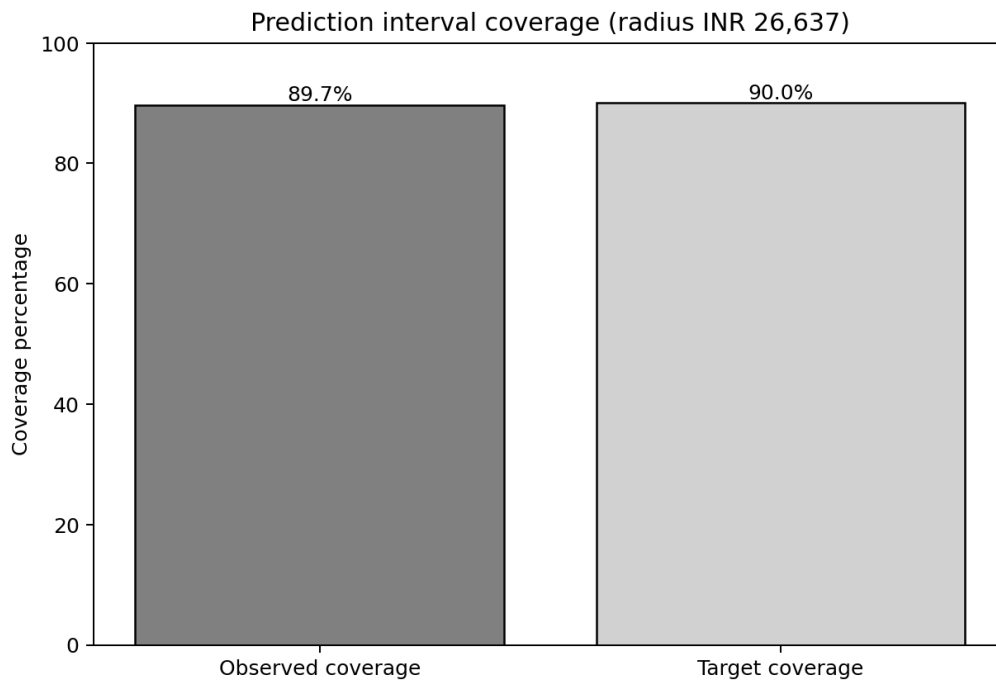


Figure 4. Observed test coverage against the 90 percent target.

What I tested: Whether the calibration-derived interval covers approximately nine of ten unseen asking rents.

What the graph shows: The fixed plus-or-minus INR 26,637 interval covers 89.7% of test listings.

Conclusion: The interval communicates uncertainty more honestly than a single point estimate, but it remains wide and non-local.

11. Experiment 5: city rent distribution

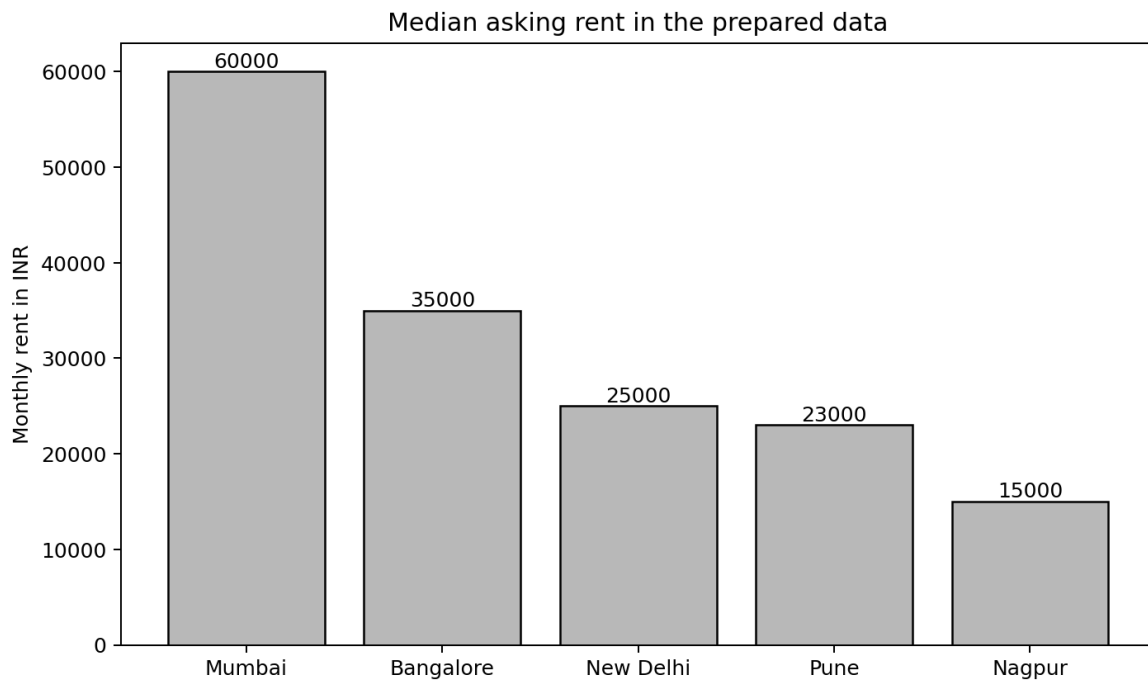


Figure 5. Median asking rent by city in the prepared dataset.

What I tested: How strongly the central rent level differs across cities before modelling other attributes.

What the graph shows: Mumbai has the highest median asking rent, while Nagpur is substantially lower.

Conclusion: City is an essential feature and a sensible baseline grouping, but locality and property characteristics are still needed.

12. Results and interpretation

On 1,831 held-out listings, MetroRent achieves MAE INR 11,652, RMSE INR 22,696, and R-squared 0.685. The city-median baseline MAE is INR 22,122.

The improvement demonstrates useful predictive structure, but asking-rent data is noisy and heavy-tailed. A large error in premium Mumbai or Bangalore listings has a much greater rupee effect than the same relative error in Nagpur.

13. Limitations and reproducibility

The sources are listing snapshots, not signed lease transactions. Duplicate advertisements may remain, property age and exact coordinates are incomplete, and market timing differs between sources. The model must not be used as a legal valuation or affordability recommendation.

The repository contains source provenance, acquisition, cleaning, training, a prediction example, metrics, tests, five figures, and this report. A future version should use time-aware collection, locality grouping, transaction data, quantile models, and interval coverage by city and price band.

14. Conclusion

I found that a structured machine-learning pipeline reduces average error substantially against a city-median baseline while calibrated intervals expose the remaining uncertainty. MetroRent demonstrates multi-source data preparation, reproducible modelling, baseline comparison, held-out evaluation, calibration, and honest geographic error analysis.